

LAPORAN IMPLEMENTASI

Pengembangan Chatbot Akademik Telkom University untuk Akses Informasi pada Dokumen Pedoman Akademik

KELOMPOK: 14

Julian Sudiyanto (103052300045)

Justin Jeremia (103052330106)

Kevin Jonathan Rotty (103052330047)

Keisha Hernantya Zahra (10305230063)

RINGKASAN

Penelitian ini mengembangkan chatbot akademik berbasis *Retrieval-Augmented Generation* (RAG) untuk membantu mahasiswa Fakultas Informatika Telkom University mengakses informasi dari dokumen pedoman akademik secara cepat dan akurat. Permasalahan utama yang diangkat adalah rendahnya keterbacaan dokumen pedoman, di mana 92,5% dari 40 responden mengalami kesulitan menemukan informasi spesifik sehingga mahasiswa cenderung bergantung pada sumber informal yang rentan terhadap misinformasi. Sistem dibangun menggunakan pipeline RAG berbasis dokumen lokal dengan *embedding model* multilingual *sentence-transformers* dan *vector store* FAISS. Pipeline terdiri dari dua tahap utama, yaitu tahap *ingestion* (preprocessing PDF ke Markdown, *chunking*, *embedding*, dan penyimpanan ke FAISS) serta tahap *inferensi* (query embedding, *similarity search*, perakitan konteks, dan generasi jawaban menggunakan Gemini-2.5-flash). Sistem difokuskan pada empat dokumen resmi, yaitu Panduan Tugas Akhir, Panduan Kerja Praktik, Pedoman Proposal TA, dan Pedoman TAK. Chatbot dikembangkan menggunakan Streamlit dan dilengkapi fitur *follow-up question generation*, *highlight* teks relevan, riwayat percakapan, tombol umpan balik, serta ekspor percakapan ke PDF untuk meningkatkan pengalaman pengguna.

Kata Kunci : *Retrieval-Augmented Generation*, FAISS, *Sentence Transformer*, Chatbot Akademik, *LangChain*, Gemini API

PENDAHULUAN

1. Overview Masalah & Solusi

Dokumen pedoman akademik merupakan acuan penting bagi mahasiswa untuk memahami setiap regulasi dan ketentuan yang ada di lingkungan universitas. Dokumen tersebut memuat ketentuan regulasi, panduan kerja praktik, kode etik, hingga syarat kelulusan. Sayangnya terdapat permasalahan utama yang membuat mahasiswa kurang bisa memahami informasi pada dokumen dengan baik. Permasalahan tersebut terletak pada sifat dokumen yang tebal dan padat, hal ini membuat intensitas mahasiswa dalam membaca pedoman resmi ini sangat rendah dan lebih memilih untuk beralih ke sumber non resmi dengan bertanya ke teman atau kakak tingkat.

Sebagai validasi atas permasalahan yang diangkat, telah dilakukan penyebaran kuisisioner kepada 40 mahasiswa, untuk mengetahui pendapat responden terhadap permasalahan ini. Berdasarkan pengumpulan data kuesioner yang telah dilakukan, sebanyak 92,5% responden menyatakan pernah mengalami kesulitan dalam mencari informasi akademik dengan kendala utama berupa dokumen yang terlalu panjang dan sulitnya menemukan bagian spesifik. Kondisi ini mendorong mahasiswa bergantung pada sumber informal seperti teman (67,5%) dan kakak tingkat (35%) yang jalurnya sangat rentan terhadap bias serta kesalahan penyampaian informasi.

Sebagai solusi, proyek Capstone ini berhasil mengimplementasikan sistem chatbot akademik cerdas berbasis pendekatan RAG. Sistem ini mampu menjembatani kebutuhan mahasiswa dalam mengakses dan memahami informasi resmi kampus secara cepat, akurat, dan otomatis selama 24 jam penuh, dengan fokus utama pada dokumen pedoman Transkrip Aktif Kemahasiswaan (TAK), panduan Kerja Praktik (KP), panduan Tugas Akhir (TA), dan panduan Proposal.

Melalui keberhasilan implementasi ini, besar harapan agar chatbot akademik ini dapat menjadi solusi yang efektif dalam meminimalkan penyebaran misinformasi, meningkatkan pemahaman regulasi secara mandiri, serta membantu meningkatkan kemudahan akses layanan informasi, tanpa terkendala batasan operasional bagi seluruh civitas akademika Telkom University.

JUSTIFIKASI ALGORITMA

1. Algoritma yang Digunakan

Sistem chatbot ini dibangun menggunakan pendekatan Retrieval-Augmented Generation (RAG), yang pertama kali diperkenalkan oleh Lewis et al. [1] sebagai solusi untuk tugas NLP yang bergantung pada pengetahuan eksternal. RAG menggabungkan dua komponen: *retriever* yang mengambil dokumen relevan dari basis pengetahuan, dan *generator* (LLM) yang menyusun jawaban berdasarkan dokumen tersebut. Berbeda dengan LLM murni, RAG menghasilkan jawaban yang *grounded* pada sumber yang dapat diverifikasi, sebuah sifat krusial untuk sistem informasi akademik.

Secara spesifik, implementasi ini menggunakan:

- Dense Retrieval berbasis cosine/L2 similarity pada ruang vektor embedding dari model paraphrase-multilingual-MiniLM-L12-v2 [2], yang mendukung pemahaman semantik lintas bahasa termasuk Bahasa Indonesia.
- FAISS (Facebook AI Similarity Search) [3] sebagai vector database persisten yang menyimpan representasi vektor dari setiap chunk dokumen.
- Gemini 2.5 Flash sebagai LLM generatif dengan suhu (temperature) 0,5 untuk menyeimbangkan konsistensi dan fleksibilitas jawaban.

2. Algoritma Alternatif & Perbandingan

Terdapat beberapa pendekatan lain yang dapat digunakan untuk membangun chatbot berbasis dokumen. Tabel berikut membandingkannya dengan RAG:

Pendekatan	Cara Kerja	Kelebihan	Kekurangan
RAG (digunakan)	Gabungkan retrieval dokumen + generasi LLM	Jawaban <i>grounded</i> , mudah diperbarui, tanpa pelatihan ulang	Bergantung pada kualitas dokumen dan akurasi retrieval
Fine-tuning LLM	Latih ulang model pada dataset domain spesifik	Jawaban sangat terspesialisasi	Mahal secara komputasi, membutuhkan data berlabel, tidak fleksibel saat dokumen berubah
Keyword Search (BM25/TF-IDF)	Cari dokumen berdasarkan kecocokan kata kunci [7]	Cepat, interpretable, tanpa model ML	Tidak memahami semantik; gagal pada sinonim, parafrase, atau pertanyaan panjang

Pure LLM Prompting	Masukkan seluruh dokumen ke konteks LLM	Implementasi sederhana	Rentan <i>hallucination</i> , terbatas oleh ukuran konteks (<i>context window</i>), tidak bisa diverifikasi [5]
--------------------	---	------------------------	---

RAG dipilih karena memberikan keseimbangan optimal antara akurasi faktual, kemudahan pembaruan dokumen, dan biaya operasional rendah, sesuai dengan kendala proyek capstone yang menggunakan layanan gratis sepenuhnya.

3. Teori Pendukung

Fondasi teoritis sistem ini bertumpu pada beberapa penelitian kunci:

- Retrieval-Augmented Generation. Lewis et al. [1] membuktikan bahwa RAG secara konsisten mengungguli model generatif murni pada tugas open-domain QA karena kemampuannya mengakses pengetahuan eksternal secara dinamis. Model tidak perlu "mengingat" fakta dalam parameternya, melainkan "mencarinya" saat dibutuhkan.
- Dense Passage Retrieval & Sentence Embeddings. Model paraphrase-multilingual-MiniLM-L12-v2 yang digunakan merupakan turunan dari arsitektur Sentence-BERT [2], yang melatih representasi kalimat menggunakan jaringan Siamese sehingga kalimat-kalimat dengan makna serupa memiliki vektor yang berdekatan dalam ruang embedding. Ini memungkinkan pencarian semantik yang melampaui kecocokan kata kunci literal.
- Similarity Search Berskala Besar. FAISS [3] menggunakan struktur indeks berbasis inverted file dan kuantisasi vektor untuk mencari k-nearest neighbors secara efisien. Pada implementasi ini, FAISS melakukan exhaustive L2 search karena jumlah dokumen masih kecil, namun arsitekturnya siap untuk diskalakan ke jutaan dokumen.
- Transformer sebagai Fondasi LLM. Gemini 2.5 Flash, model yang digunakan sebagai generator, dibangun di atas arsitektur Transformer [4] yang memungkinkan pemahaman konteks panjang dan generasi teks yang koheren dari prompt yang kompleks.
- Implementasi serupa juga telah dilaporkan dalam survei Gao et al. [8], yang mengonfirmasi bahwa pola RAG dengan dense retriever + generative LLM merupakan arsitektur dominan untuk sistem QA berbasis dokumen pada tahun 2023–2024.

ALUR IMPLEMENTASI

1. Diagram Blok

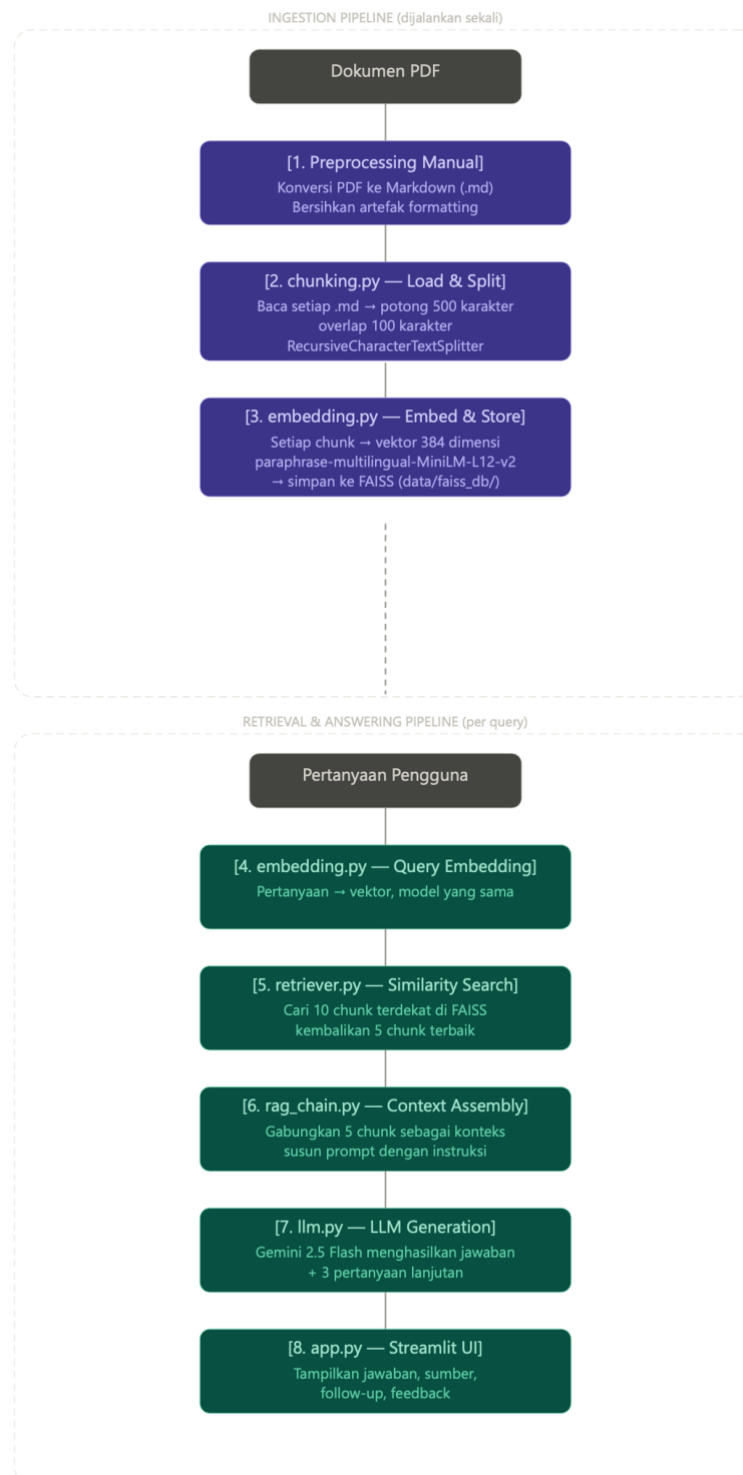


Gambar 1. Flowchart metodologi

Gambar 1. menunjukkan arsitektur sistem chatbot akademik berbasis Retrieval-Augmented Generation (RAG) yang digunakan pada penelitian ini. Metode yang diterapkan masih mengikuti rancangan utama pada proposal, yaitu menggunakan pipeline RAG yang terdiri dari tahap ingestion dan tahap retrieval & answering, namun mengalami beberapa penyesuaian implementasi dan penggantian tools selama proses pengembangan. Pada tahap ingestion, dokumen akademik diproses melalui preprocessing PDF, konversi ke Markdown, chunking, pembuatan embedding menggunakan model sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2, serta penyimpanan vektor ke dalam FAISS sebagai vector store. Selanjutnya pada tahap retrieval & answering, pertanyaan pengguna diubah menjadi query embedding, dilakukan similarity search untuk mengambil chunk paling relevan, kemudian konteks dikirim ke Gemini-2.5-flash untuk menghasilkan jawaban yang ditampilkan melalui antarmuka Streamlit.

2. Detail Tahapan

Sistem bekerja dalam dua pipeline terpisah, sebagaimana diilustrasikan pada gambar metodologi. Alur pada gambar 2. di bawah menggambarkan bagaimana metologi pada gambar 1. diimplementasikan melalui struktur project yang modular:



Gambar 2. Alur implementasi

3. Penjelasan Kode

Implementasi kode program pada proyek ini dibagi ke dalam beberapa modul agar sistem lebih terstruktur dan mudah dikembangkan. Berikut adalah penjelasan untuk masing-masing modul kode:

- **src/chunking.py (Preprocessing & Pemecahan Dokumen)**

Modul ini membaca seluruh file Markdown dari folder `cleaned_file/` dan memecahnya menjadi unit teks yang lebih kecil (chunk). Pemecahan menggunakan `RecursiveCharacterTextSplitter` dari `LangChain` dengan ukuran chunk 500 karakter dan overlap 100 karakter. Strategi overlap penting agar informasi yang berada di perbatasan dua chunk tidak hilang saat retrieval.

```
splitter = RecursiveCharacterTextSplitter(  
    chunk_size=500,  
    chunk_overlap=100,  
    separators=["\n\n", "\n", ". ", " ", ""],  
)
```

Chunk yang terlalu pendek (< 80 karakter) atau hanya berisi simbol Markdown dibuang. Setiap chunk yang tersisa mendapat prefiks nama dokumen sumbernya (mis. [Pedoman TAK]) agar model embedding dapat mengaitkan konten dengan sumbernya secara lebih kuat.

- **src/embedding.py (Pembuatan Vektor & Penyimpanan FAISS)**

Modul ini mengonversi setiap chunk teks menjadi vektor numerik menggunakan model `paraphrase-multilingual-MiniLM-L12-v2` dari `HuggingFace`. Model ini menghasilkan vektor 384 dimensi yang merepresentasikan makna semantik teks. Seluruh vektor beserta teks aslinya disimpan secara persisten ke disk melalui `FAISS.save_local()`.

```
vectorstore = FAISS.from_documents(documents=chunks,  
    embedding=embedding_fn)  
vectorstore.save_local("data/faiss_db")
```

Fungsi `load_vectorstore()` digunakan saat inferensi untuk memuat kembali indeks FAISS tanpa perlu membangun ulang dari awal.

- **src/retriever.py (Pencarian Konteks Relevan)**

Pada setiap query, pertanyaan pengguna diubah ke vektor menggunakan model embedding yang sama (konsistensi ini kritis, menggunakan model berbeda akan menghasilkan vektor yang tidak kompatibel). FAISS kemudian menghitung jarak L2 antara vektor query dengan seluruh vektor chunk dalam indeks.

```
TOP_K = 10 # kandidat awal
```

```
RETURN_K = 5 # chunk yang dikirim ke LLM
docs = vectorstore.similarity_search(question, k=TOP_K)
result = docs[:RETURN_K]
```

Strategi dua-tahap (ambil 10, kembalikan 5) memberi ruang untuk memilih chunk yang benar-benar relevan sambil menghindari noise dari chunk dengan relevansi rendah.

- **src/llm.py (Koneksi ke Gemini API)**

Modul ini menginisialisasi objek LLM ChatGoogleGenerativeAI dari LangChain yang terhubung ke model gemini-2.5-flash. Nilai temperature=0.5 dipilih untuk menghasilkan jawaban yang konsisten namun tetap memiliki variasi bahasa yang natural, nilai lebih rendah menghasilkan jawaban terlalu kaku, nilai lebih tinggi meningkatkan risiko hallucination.

- **src/rag_chain.py (Perakitan Pipeline RAG)**

Ini adalah modul inti yang menggabungkan seluruh komponen. Fungsi build_prompt() menyusun konteks dari 5 chunk terbaik ke dalam format bernomor, lalu menggabungkannya dengan pertanyaan pengguna dalam satu prompt terstruktur. Prompt secara eksplisit meminta model untuk menjawab hanya berdasarkan konteks, jujur saat informasi tidak tersedia, dan menghasilkan 3 pertanyaan lanjutan dalam format yang dapat di-parse.

Kamu adalah asisten akademik Telkom University.

Jawab berdasarkan informasi dalam konteks di bawah ini...

Setelah memberikan jawaban, tambahkan tepat 3 pertanyaan lanjutan:

PERTANYAAN_LANJUTAN:

1. [pertanyaan 1]

...

Fungsi parse_response() kemudian memisahkan jawaban utama dari blok pertanyaan lanjutan menggunakan delimiter PERTANYAAN_LANJUTAN:, menghasilkan tuple (answer_text, follow_ups).

- **app.py (Antarmuka Streamlit)**

Aplikasi Streamlit mengelola seluruh interaksi pengguna. Setiap percakapan disimpan di st.session_state sebagai daftar message berstruktur {"role", "content", "sources", "follow_ups", "feedback"}. Variabel active_conv_idx melacak apakah sesi aktif merupakan percakapan baru (None) atau percakapan yang dimuat dari riwayat (indeks integer), sehingga navigasi antar riwayat tidak menyebabkan duplikasi. Setelah LLM mengembalikan jawaban, st.rerun() dipanggil secara eksplisit untuk memaksa Streamlit me-render ulang seluruh halaman, ini diperlukan agar tombol feedback dan tombol unduh PDF di sidebar muncul segera setelah percakapan pertama.

FITUR PRODUK

1. Fitur Utama & Cara Kerjanya

- Tanya Jawab Berbasis Dokumen Resmi

Pengguna dapat mengajukan pertanyaan dalam Bahasa Indonesia terkait empat dokumen akademik resmi. Chatbot menjawab berdasarkan **chunk** yang paling relevan secara semantik, bukan berdasarkan tebakan model. Jika informasi tidak ditemukan dalam dokumen, sistem secara eksplisit menyatakan keterbatasannya daripada mengarang jawaban.

- Suggested Follow-up Questions

Setiap jawaban dilengkapi dengan tiga pertanyaan lanjutan yang dihasilkan otomatis oleh LLM dan relevan dengan topik yang sedang dibahas. Pengguna dapat mengklik salah satu pertanyaan untuk langsung mengajukannya tanpa mengetik ulang. Fitur ini mendorong eksplorasi informasi yang lebih mendalam.

- Highlight Teks Relevan dari Sumber

Di bawah setiap jawaban, terdapat *expander* "Sumber dokumen" yang menampilkan potongan teks (*snippet*) dari dokumen asli yang digunakan sebagai konteks. Kata kunci dari pertanyaan pengguna di-*highlight* secara otomatis menggunakan regex dan HTML inline, memudahkan pengguna memverifikasi asal informasi jawaban.

- Feedback Jawaban (👍 / 👎)

Setiap jawaban memiliki tombol umpan balik positif dan negatif. Data umpan balik disimpan ke ``data/feedback.json`` dengan *timestamp*, pertanyaan, dan pratinjau jawaban. Data ini dapat digunakan sebagai bahan evaluasi kualitatif model di masa mendatang.

- Riwayat Percakapan

Seluruh percakapan yang telah dilakukan tersimpan di *sidebar* dengan judul otomatis berdasarkan pertanyaan pertama. Pengguna dapat berpindah antar riwayat tanpa kehilangan pesan; sistem menyimpan *state* percakapan aktif sebelum berpindah. Tombol "Percakapan Baru" memulai sesi kosong dan mengarsipkan percakapan yang sedang berjalan.

- Ekspor Percakapan ke PDF

Pengguna dapat mengunduh seluruh percakapan aktif sebagai file PDF satu klik melalui tombol "Export PDF" di *sidebar*. PDF berisi *timestamp** ekspor, seluruh pasangan pertanyaan-jawaban dengan label visual yang jelas, dan menggunakan format yang bersih tanpa *markdown syntax*.

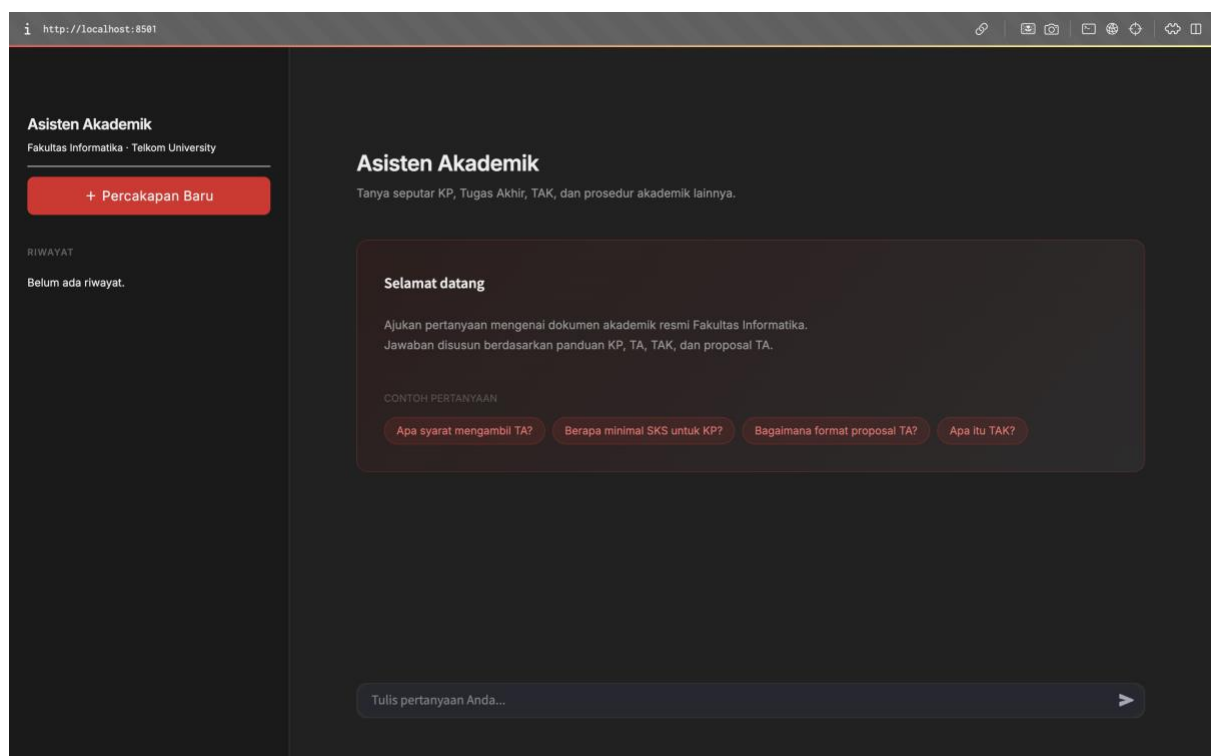
- **Animasi Ketik (*Typing Animation*)**

Jawaban ditampilkan secara bertahap kata per kata dengan jeda 20ms menggunakan `'st.empty()'` dan loop, meniru efek mengetik untuk meningkatkan keterbacaan dan pengalaman pengguna.

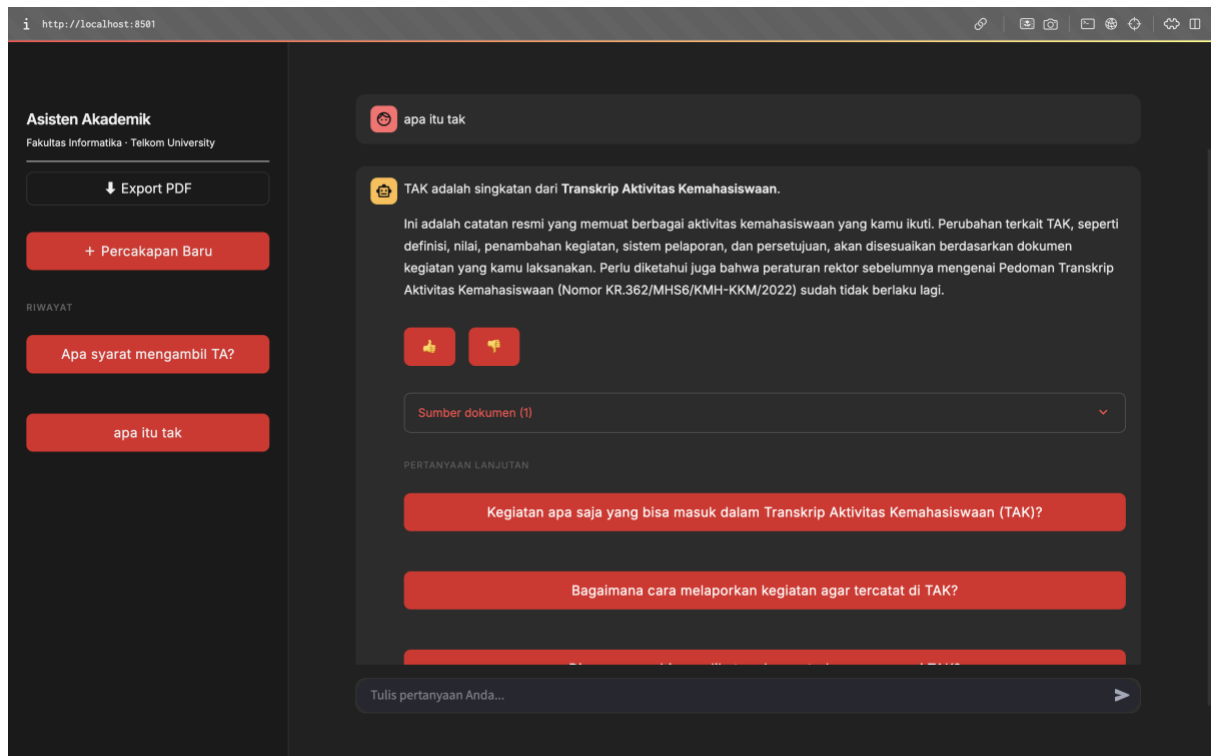
- **Welcome Card & Contoh Pertanyaan**

Saat percakapan masih kosong, halaman menampilkan kartu sambutan berisi deskripsi singkat sistem dan empat contoh pertanyaan dalam bentuk *chip* yang dapat diklik. Ini membantu pengguna baru memahami kapabilitas sistem tanpa perlu membaca dokumentasi.

2. Visualisasi Output



Gambar 3. Tampilan awal aplikasi



Gambar 4. Tampilan lanjut aplikasi

Gambar di atas menunjukkan antarmuka chatbot akademik berbasis RAG yang dikembangkan menggunakan Streamlit. Sistem menyediakan fitur percakapan interaktif untuk membantu mahasiswa mencari informasi dari dokumen resmi akademik seperti Panduan TA, KP, Proposal TA, dan TAK. Gambar 3. menunjukkan aplikasi saat pertama kali dibuka, terdapat welcome card dan sidebar di samping untuk riwayat percakapan. Gambar 4. menunjukkan tampilan aplikasi saat user mulai bertanya, terdapat feedback untuk jawaban chatbot, sumber jawaban, pertanyaan lanjutan, riwayat chat, dan tombol untuk ekspor percakapan menjadi PDF.

RENCANA EVALUASI

1. Rencana Evaluasi

Evaluasi dirancang dalam dua lapisan: evaluasi otomatis untuk mengukur kualitas pipeline RAG secara kuantitatif, dan evaluasi manusia untuk mengukur kepuasan dan kegunaan nyata di lapangan.

- Lapisan 1: Evaluasi Otomatis dengan RAGAS

RAGAS (*Retrieval-Augmented Generation Assessment*) [6] adalah *framework* evaluasi yang dirancang khusus untuk pipeline RAG. Evaluasi dilakukan dengan menyiapkan dataset uji berisi 20–30 pasangan (pertanyaan, jawaban referensi, konteks ground truth) yang dibuat secara manual dari dokumen sumber.

Dataset uji mencakup variasi pertanyaan: Pertanyaan faktual langsung, Pertanyaan procedural, Pertanyaan yang jawabannya tidak ada dalam dokumen (untuk mengukur kejujuran sistem) Pertanyaan ambigu atau tidak lengkap

- Lapisan 2: Evaluasi Pengguna

Melibatkan 10–15 mahasiswa Fakultas Informatika untuk menggunakan sistem secara langsung dan mengisi kuesioner penilaian menggunakan skala Likert 1–5 pada dimensi akurasi, kejelasan, relevansi, dan kemudahan penggunaan.

2. Metrik yang Digunakan

- Metrik Pipeline RAG (RAGAS)

Metrik	Definisi	Target
Faithfulness	Sejauh mana klaim dalam jawaban dapat didukung oleh konteks yang diambil. Nilai 1.0 berarti tidak ada klaim yang "dikarang".	≥ 0.80
Answer Relevancy	Seberapa relevan jawaban dengan pertanyaan yang diajukan. Diukur menggunakan cosine similarity antara pertanyaan asli dan pertanyaan yang diregenerasi dari jawaban.	≥ 0.75
Context Recall	Proporsi informasi dalam jawaban referensi yang dapat ditemukan di dalam konteks yang diambil oleh <i>retriever</i> . Mengukur apakah <i>retriever</i> berhasil menemukan dokumen yang tepat.	≥ 0.70
Context Precision	Proporsi <i>chunk</i> yang diambil yang benar-benar digunakan dalam jawaban. Mengukur efisiensi retrieval, apakah <i>chunk</i> yang tidak relevan ikut terambil.	≥ 0.65

- **Metrik Evaluasi Pengguna**

Dimensi	Instrumen	Cara Ukur
Akurasi Jawaban	Kuesioner Likert 1–5	Apakah jawaban sesuai dengan dokumen resmi?
Kejelasan Bahasa	Kuesioner Likert 1–5	Apakah jawaban mudah dipahami?
Relevansi Jawaban	Kuesioner Likert 1–5	Apakah jawaban menjawab pertanyaan secara tepat?
Kemudahan Penggunaan	System Usability Scale (SUS)	Skor SUS agregat dari 10 pertanyaan standar
Tingkat Penerimaan	Biner (Ya/Tidak)	Apakah pengguna akan menggunakan sistem ini sebagai referensi akademik?

DAFTAR PUSTAKA

- [1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-T. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020.
- [2] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proc. 2019 Conf. on Empirical Methods in Natural Language Processing and the 9th Int. Joint Conf. on Natural Language Processing (EMNLP-IJCNLP)*, 2019, pp. 3982–3992.
- [3] J. Johnson, M. Douze, and H. Jégou, "Billion-Scale Similarity Search with GPUs," *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535–547, Sep. 2021.
- [4] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention Is All You Need," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.
- [5] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell et al., "Language Models are Few-Shot Learners," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 1877–1901, 2020.
- [6] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "RAGAS: Automated Evaluation of Retrieval Augmented Generation," in *Proc. 18th Conf. of the European Chapter of the Association for Computational Linguistics (EACL)*, 2024, pp. 150–158.
- [7] S. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," *Found. Trends Inf. Retr.*, vol. 3, no. 4, pp. 333–389, 2009.
- [8] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, and H. Wang, "Retrieval-Augmented Generation for Large Language Models: A Survey," *arXiv preprint arXiv:2312.10997*, 2023.
- [9] H. Chase, "LangChain: Building applications with LLMs through composability," GitHub, 2022. [Online]. Available: <https://github.com/langchain-ai/langchain>
- [10] Google DeepMind, "Gemini: A Family of Highly Capable Multimodal Models," *arXiv preprint arXiv:2312.11805*, 2023.